NOMS et PRENOMS : ***DATE : 23/10/2026***
-David WATEBA
-Antoine PERFETTINI

Rapport Smart Home STM32

Objectifs

*L'objectif de ce mini-projet était de concevoir le programme embarqué d'une maison intelligente, en remplaçant l'ESP32 de la maquette Keyestudio par une carte **NUCLEO-F411RE**. Il fallait :*

- *acquérir les valeurs des capteurs à intervalles réguliers (gaz, pluie, température)*
- *gérer les modes **HOME** et **AWAY** et détecter une intrusion avec le PIR*
- *déclencher des alarmes visuelles et sonores*
- *superviser la maison depuis un PC par l'**UART***
- *piloter une fenêtre motorisée à partir d'un bouton et d'un capteur de pluie*
- *écrire un driver **I²C** pour le **TMP102** à partir de sa datasheet*
- *réunir le tout dans une seule application*

1. Architecture et brochage

1.1 Matériel

*Nous avons utilisé la carte **NUCLEO-F411RE**, programmée en C avec la HAL sous **STM32CubeIDE**, et les modules du kit KS5009 (PIR, buzzer passif, LED, capteur de gaz, capteur de pluie, bouton, servo), plus un capteur de température **TMP102**. Le bouton B1, la LED LD2 et la liaison série vers le PC sont déjà sur la carte.*

*Sur la breadboard, nous avons séparé un rail **5V** et un rail **3,3V**, avec une masse commune. Chaque module est branché sur le rail qu'impose sa datasheet :*

- *le capteur de gaz en 5V, car sa résistance de chauffe l'exige*
- *le servo en 5V, il tire un fort courant au démarrage*
- *le buzzer en 5V*
- *le TMP102 obligatoirement en 3,3V (plage de 1,4 à 3,6V)*
- *le PIR, la LED, le bouton et le capteur de pluie en 3,3V*

1.2 Architecture logicielle

*Toutes nos interruptions (timers, boutons, PIR) se contentent de lever un drapeau, par exemple **mesureGaz = 1**. Le vrai travail (ADC, I²C, UART, décisions) est fait dans la boucle while(1), qui teste les drapeaux. Comme ça, une interruption ne bloque jamais les autres. Les variables partagées entre les interruptions et la boucle sont déclarées **volatile**, pour que le compilateur relise toujours leur vraie valeur.*

Section du main.c	*Contenu*
USER CODE PD	*les **#define** : seuils du gaz et de la pluie, positions du servo, adresse et registres du TMP102*
USER CODE PV	*les **variables** : drapeaux volatile, mode, alarmes, mesures, état de la fenêtre*
USER CODE 0	*nos **fonctions** : envoyer(), ouvrirFenetre(), fermerFenetre() et les 4 fonctions du driver TMP102*
USER CODE 2	*le **démarrage** : timers, PWM, fenêtre fermée, test du TMP102, message de bienvenue*
USER CODE 3	*la **boucle principale** : traitement de chaque drapeau*
USER CODE 4	*les **callbacks** HAL_TIM_PeriodElapsedCallback() et HAL_GPIO_EXTI_Callback()*

1.3 Horloge

*L'oscillateur interne à 16 MHz est monté à **100 MHz**. Tous nos timers sont cadencés à 100 MHz, d'où la formule utilisée partout :*

$$T = (PSC + 1) \times (ARR + 1) / 100\ MHz$$

1.4 Brochage et périphériques

*Pour choisir les broches, nous sommes partis du périphérique dont nous avions besoin (TIM3_CH1, TIM4_CH1, ADC1_IN0/IN1, I2C1). Nous avons ensuite pris la broche qui le porte sur le connecteur Arduino, en vérifiant dans l'**UM1724** (la documentation de la carte) qu'elle n'était pas déjà utilisée.*

Composant	Broche	Connecteur Arduino	Périphérique	Acquisition
Bouton B1 (HOME/AWAY)	PC13	—	GPIO_EXTI13, front descendant	Interruption
PIR	PB5	D4	GPIO_EXTI5, front montant, pull-down	Interruption
LED d'alarme (commune)	PA8	D7	GPIO Output push-pull	Sortie
Buzzer passif	PB4	D5	TIM3_CH1, PWM en 2 kHz	Sortie PWM
Capteur de gaz	PA0	A0	ADC1_IN0 (rang 1)	Scrutation toutes les 3 s
Capteur de pluie	PA1	A1	ADC1_IN1 (rang 2)	Scrutation toutes les 3 s
Liaison PC	PA2 / PA3	— (ST-LINK)	USART2, 115200 bauds	Sortie, envoi bloquant
Servo fenêtre	PB6	D10	TIM4_CH1, PWM en 50 Hz	Sortie PWM
Bouton fenêtre	PA10	D2	GPIO_EXTI10, front montant, pull-down	Interruption
TMP102 SCL / SDA	PB8 / PB9	D15 / D14	I2C1, 100 kHz	Scrutation toutes les 3 s

*Le buzzer et le servo sont sur deux timers différents parce que les canaux d'un même timer partagent la même période : on ne peut pas avoir 2 kHz et 50 Hz sur le même timer. B1 et le bouton fenêtre partagent l'interruption **EXTI15_10**, et notre callback les distingue grâce au paramètre GPIO_Pin.*

N'ayant pas pris de photos du montage, nous avons refait les schémas de câblage des figures avec l'outil Artifact de Claude, à partir de notre brochage réel.

2. Scénarios réalisés

2.1 Scénario 1 : base temporelle

*Il fallait une interruption périodique de 2 à 5 s pour cadencer les mesures, nous avons pris **3 s**. Nous avons utilisé **TIM11**, un timer simple sans canal de sortie, ce qui suffit pour une base de temps et laisse TIM3 et TIM4 libres pour les PWM. Son interruption est activée dans le NVIC.*

*Avec **PSC = 9999**, les 100 MHz deviennent 100 MHz / 10 000 = **10 kHz**, soit un top toutes les 0,1 ms. Avec **ARR = 29999**, le compteur déborde tous les 30 000 tops, soit **3 s**. Sans prescaler, il faudrait compter 300 millions de tops, ce qui est impossible sur 16 bits.*

*Le timer est démarré dans **USER CODE 2** avec HAL_TIM_Base_Start_IT(). Dans la callback (**USER CODE 4**), on vérifie que c'est bien TIM11 qui a débordé et on lève le drapeau mesureGaz. Nous ne mettons pas toutes les opérations dans la callback parce qu'elle doit s'exécuter le plus vite possible.*

```
357 // Appelée par la HAL à chaque débordement de timer. TIM10 et TIM11 passent
358 // tous les deux par ici, donc on regarde lequel a débordé.
359 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
360 {
361   if (htim->Instance == TIM11)
362   {
363     mesureGaz = 1;                // on lève juste le drapeau, la mesure se fait dans la boucle
364   }
365   else if (htim->Instance == TIM10)
366   {
367     if (alarme == 1 || alarmeGaz == 1)   // la LED ne clignote que si une alarme est active
368       HAL_GPIO_TogglePin(LED_ALARME_GPIO_Port, LED_ALARME_Pin);   // une seule LED pour les deux alarmes
369   }
370 }
```

Extrait du main.c, lignes 357 à 370

*Pour vérifier la période, la callback faisait au départ basculer la LED **LD2 (PA5)** : elle changeait d'état toutes les 3 s. De plus, à l'oscilloscope sur PA5, nous avons observé un signal carré de période **6 s**, soit une bascule toutes les 3 s.*



Figure 1 : Scénario 1, LED LD2 et TIM11, sans câblage externe

2.2 Scénario 2 : modes HOME/AWAY et intrusion

Ici, B1 fait basculer entre HOME et AWAY, et en AWAY un mouvement détecté par le PIR doit déclencher une alarme sonore et visuelle sans bloquer le programme. Nous avons réglé les broches et les timers ainsi :

- ***B1 sur PC13** : EXTI13, front descendant, sans pull, car B1 est actif à l'état bas avec une pull-up sur la carte*
- ***PIR sur PB5** : EXTI5, front montant, sa sortie passe de 0 à 3 V lors d'un mouvement, avec une pull-down pour éviter une entrée flottante*
- ***LED d'alarme sur PA8** en sortie push-pull, une seule LED pour le gaz et l'intrusion (choix validé avec l'enseignante le 15/10, l'UART indiquant quelle alarme est active)*
- ***TIM10** : PSC = 9999 et ARR = 2499, débordement toutes les **250 ms**, donc clignotement à **2 Hz***
- ***Buzzer passif sur TIM3_CH1 (PB4)** : PSC = 99 donne 1 MHz, ARR = 499 donne **2 kHz**, Pulse = 250 donne **50 %***

Un buzzer passif ne sonne qu'avec un signal alternatif : c'est la PWM qui le produit, toute seule, sans occuper le processeur.

*Dans la callback EXTI (**USER CODE 4**), un anti-rebond de **200 ms** basé sur HAL_GetTick() filtre les appuis sur B1. Nous n'avons pas utilisé HAL_Delay(), qui bloquerait tout le système à l'intérieur de l'interruption.*

```c
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)   // GPIO_Pin indique quelle broche a déclenché
{
  if (GPIO_Pin == B1_Pin)
  {
    if (HAL_GetTick() - dernierAppui > 200)   // anti-rebond : on ignore un appui à moins de 200 ms du précédent
    {
      dernierAppui = HAL_GetTick();
      boutonAppuye = 1;
    }
  }

  else if (GPIO_Pin == PIR_Pin)
  {
    mouvement = 1;   // on signale le mouvement, le traitement se fait dans la boucle
  }
```

Extrait du main.c, lignes 372 à 386

*Dans la boucle (**USER CODE 3**), un mouvement en AWAY démarre le buzzer, met alarme à 1 et envoie INTRUSION. L'alarme reste active jusqu'au retour en HOME, et TIM10 ne fait basculer la LED que si une alarme est en cours.*

```c
        if (mouvement == 1)   // le PIR a détecté quelqu'un
        {
          mouvement = 0;

          if (mode == AWAY && alarme == 0)   // l'alarme ne part qu'en AWAY, et une seule fois
              {
                  alarme = 1;
                  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);   // le buzzer se met à sonner
                  envoyer("INTRUSION\r\n");
              }
        }
```

Extrait du main.c, lignes 231 à 241

*Sur la maquette, en AWAY, le passage de la main devant le PIR affiche **INTRUSION** avec le buzzer et la LED qui clignote, et le retour en HOME coupe l'alarme. Nous avons aussi remarqué que le PIR détecte souvent la personne qui vient d'appuyer sur B1.*

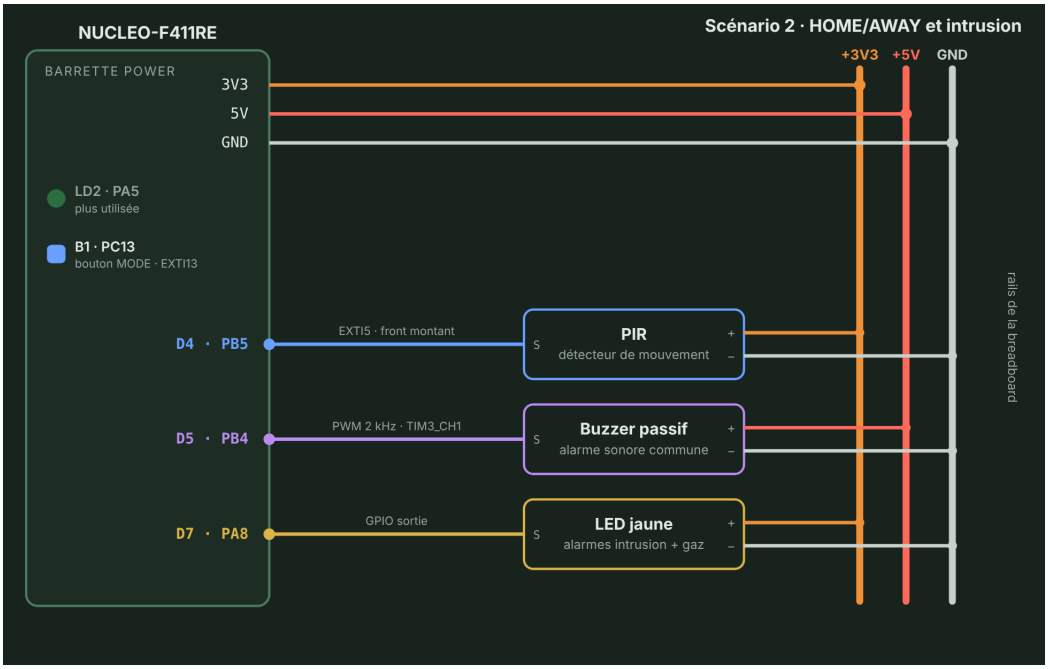


Figure 2 : Câblage du scénario 2 (PIR, LED d'alarme, buzzer)

```
Mode AWAY
INTRUSION
Fenetre bloquee
Gaz = 722 Pluie = 434
Temp = 23.5 C
Mode HOME
```

Figure 3 : Terminal série, passage en AWAY, intrusion, puis retour en HOME

2.3 Scénario 3 : détection de gaz

*Le gaz est mesuré toutes les 3 s et doit déclencher une alarme au-delà d'un seuil, dans les deux modes. Le **MQ-4** doit être alimenté en 5V pour sa chauffe, et sa sortie peut monter jusqu'à 5V, alors que l'ADC n'accepte que 3,3V. Nous avons donc ajouté un **pont diviseur 22 kΩ / 10 kΩ** (× 0,31) : le 5V deviennent au maximum **1,56 V** sur PA0.*

Côté CubeMX, l'ADC1 est en 12 bits et en mode scan avec deux rangs, IN0 pour le gaz et IN1 pour la pluie. Nous avons choisi :

- *un temps d'échantillonnage de **480 cycles**, car le pont présente une forte impédance*
- *l'EOC à chaque conversion, pour lire les deux valeurs l'une après l'autre*
- *une horloge ADC de 100 MHz / 4 = **25 MHz**, soit environ **20 µs** par conversion qui est < **36 MHz** max de l'ADC*

Les seuils ne viennent pas de la documentation : nous les avons fixés à partir de nos propres mesures sur la maquette.

Situation	*Valeur ADC*
Repos (selon la chauffe)	*430 à 650*
Souffle	*1000 à 1100*
Gaz de briquet	*1450 à 1580*
Seuil de déclenchement	*1250*
Seuil de fin d'alarme	*1050*

*Nous avons pris deux seuils : l'alarme se déclenche au-dessus de **1250** et ne s'arrête que sous **1050**. Avec un unique seuil, une valeur qui oscille autour de 1250 ferait clignoter l'alarme toutes les 3 s.*

*Les seuils sont dans **USER CODE PD**, la lecture de l'ADC et la décision dans **USER CODE 3**. La décision ne teste pas le mode, ce qui rend la surveillance permanente. Comme la LED et le buzzer sont partagés, la fin de l'alarme gaz ne les coupe que s'il n'y a pas d'intrusion en cours.*

```
259             HAL_ADC_Start(&hadc1);
260             HAL_ADC_PollForConversion(&hadc1, 10);
261             valeurGaz = HAL_ADC_GetValue(&hadc1);     // rang 1 : capteur de gaz
262             HAL_ADC_PollForConversion(&hadc1, 10);
263             valeurPluie = HAL_ADC_GetValue(&hadc1);   // rang 2 : capteur de pluie
264             HAL_ADC_Stop(&hadc1);
```

Extrait du main.c, lignes 259 à 264

```
287         if (valeurGaz > SEUIL_GAZ_ON && alarmeGaz == 0)   // surveillé dans les deux modes
288             {
289               alarmeGaz = 1;
290               HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);   // même buzzer pour les deux alarmes
291               envoyer("GAS ALERT\r\n");
292             }
293         else if (valeurGaz < SEUIL_GAZ_OFF && alarmeGaz == 1)
294         {
295           alarmeGaz = 0;
296           if (alarme == 0)   // plus aucune alarme : on coupe tout
297           {
298             HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_1);
299             HAL_GPIO_WritePin(LED_ALARME_GPIO_Port, LED_ALARME_Pin, GPIO_PIN_RESET);
300           }
301           envoyer("Gaz normal\r\n");
302         }
```

Extrait du main.c, lignes 287 à 302

*Avec le gaz d'un briquet (sans flamme), la valeur dépassait 1450 et **GAS ALERT** s'affichait, en HOME comme en AWAY. Une fois le gaz dissipé, « Gaz normal » apparaissait.*

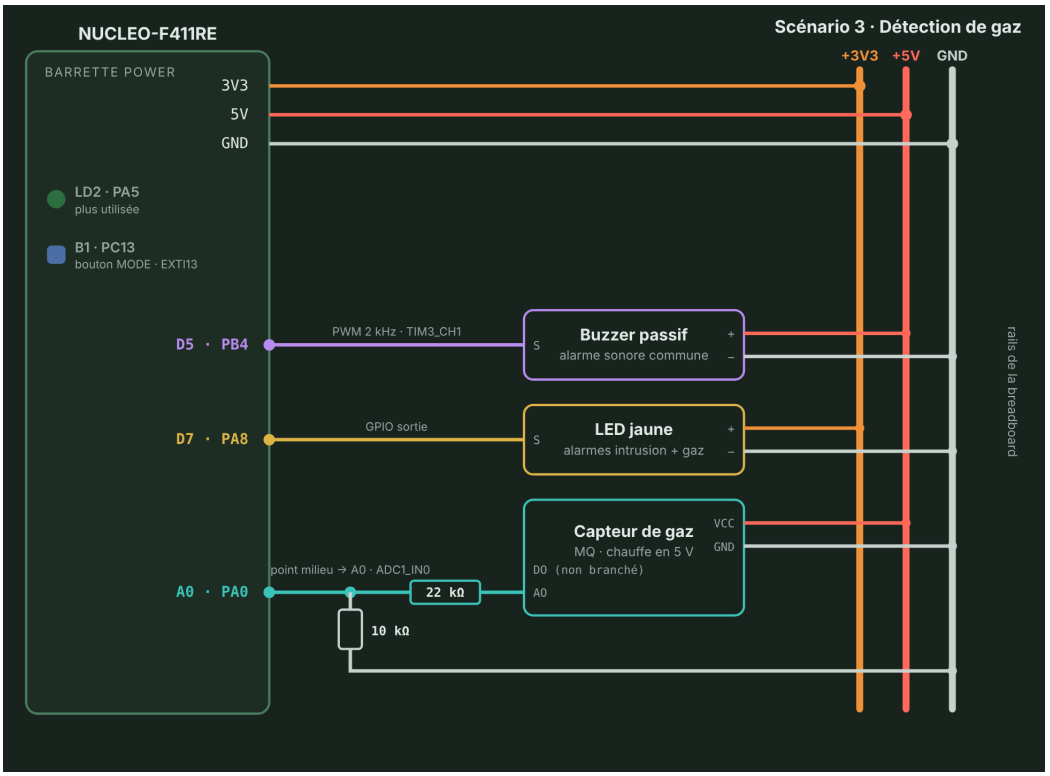


Figure 4 : Câblage du scénario 3 (capteur de gaz et pont diviseur)

2.4 Scénario 4 : supervision UART

*Le but içi était d'afficher un message de bienvenue, puis les mesures et l'état de la maison sur un terminal PC. Nous avons choisi l'**USART2** (PA2/PA3) car, d'après l'UM1724, elle est reliée au ST-LINK, qui la présente au PC comme un port COM virtuel par le câble USB. Elle est réglée à **115200 bauds, 8N1**.*

*Pour des raisons de praticité et de lisibilité du code, nous avons décidé d'écrire une seule fonction **envoyer()**, dans **USER CODE 0** qui sert à tout le programme, et les valeurs sont mises en texte avec **sprintf()**. L'envoi est bloquant, mais il est fait uniquement dans la boucle principale et ne dure qu'environ 2 ms par ligne.*

```
93  void envoyer(char *texte)        // envoie un texte au PC par l'UART2
94  {
95    HAL_UART_Transmit(&huart2, (uint8_t*)texte, strlen(texte), 100);
96  }
```

Extrait du main.c, lignes 93 à 96

Au premier essai, rien ne s'affichait : le terminal était ouvert sur le mauvais port COM, Windows n'attribuant pas le même numéro d'un PC à l'autre (COM4 sur l'un, COM7 sur l'autre). Il a suffi de régler le terminal sur le bon port, sans toucher au code. Ensuite, les mesures sont arrivées toutes les 3 s. Nous n'avons pas utilisé le LCD, l'énoncé demandant « UART et/ou LCD ».

```
Gaz = 728 Pluie = 467
Temp = 24.1 C
Gaz = 650 Pluie = 363
Temp = 24.1 C
Gaz = 1157 Pluie = 867
Temp = 24.1 C
```

Figure 5 : Terminal série, mesures envoyées toutes les 3 secondes

2.5 Scénario 5 : gestion de la fenêtre

*La fenêtre est actionnée par un servo moteur. Elle s'ouvre et se ferme avec un bouton. Elle se ferme toute seule s'il pleut et reste fermée en AWAY et son état est signalé par **WINDOW OPEN** et **WINDOW CLOSED**.*

*Le servo attend une impulsion toutes les **20 ms** (50 Hz), et c'est la largeur de cette impulsion qui fixe l'angle. Nous l'avons alimenté sur le rail 5V et commandé par **TIM4_CH1** sur PB6 (D10) :*

- ***PSC = 99*** *donne 1 MHz, donc 1 top = 1 µs et CCR1 s'écrit directement en microsecondes*
- ***ARR = 19999*** *donne 20 000 µs = 20 ms, soit* ***50 Hz***
- ***CCR1 = 500*** *pour la fenêtre fermée,* ***CCR1 = 1500*** *pour la fenêtre ouverte*

Ces deux positions viennent de nos essais et pas de la documentation : en debug, nous avons modifié CCR1 dans la vue des registres jusqu'à obtenir les deux positions de la fenêtre. Le même signal ne donne pas le même angle d'un modèle de servo à l'autre.

Le ***bouton fenêtre*** *est sur* ***PA10*** *(****D2****), en* ***EXTI10****,* ***front montant****,* ***pull-down****, avec le même anti-rebond de* ***200 ms*** *que* ***B1****. En interruption, on ne rate aucun appui.*

Le ***capteur de pluie*** *est sur PA1 (rang 2 de l'ADC). Alimenté en 3,3V, sa sortie ne peut pas dépasser 3,3V, nous l'avons donc branché directement, sans pont diviseur. D'après nos mesures, il donne environ* ***100*** *à sec et jusqu'à* ***4095*** *mouillé, et nous avons fixé le seuil à* ***2000*** *pour qu'il soit loin des deux états.*

Les positions sont définies dans ***USER CODE PD*** *(POS_OUVERTE, POS_FERMEE). Les fonctions* ***ouvrirFenetre()*** *et* ***fermerFenetre()*** *dans* ***USER CODE 0*** *écrivent la largeur voulue dans CCR1 avec la macro* ***__HAL_TIM_SET_COMPARE()****. La PWM est lancée une seule fois au démarrage car le servo a besoin du signal en continu pour tenir sa position.*

```
97  /* ouvre la fenêtre : on change juste la largeur d'impulsion du servo */
98  void ouvrirFenetre(void)
99  {
100   __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, POS_OUVERTE); // écrit la valeur dans CCR1 de TIM4
101   fenetreOuverte = 1;
102   envoyer("WINDOW OPEN\r\n");
103 }
104 /* ferme la fenêtre */
105 void fermerFenetre(void)
106 {
107   __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, POS_FERMEE);
108   fenetreOuverte = 0;
109   envoyer("WINDOW CLOSED\r\n");
110 }
```

Extrait du main.c, lignes 97 à 110

*Dans la boucle principale (****USER CODE 3****), trois règles s'appliquent :*

- *un appui sur le bouton en AWAY ou sous la pluie affiche « Fenetre bloquee », sinon il ouvre ou ferme la fenêtre*
- *le début de la pluie ferme la fenêtre, et sa fin débloque le bouton sans rouvrir. Dans le code nous avons préféré ne pas rouvrir une fenêtre que l'utilisateur n'a pas redemandée*
- *le passage en AWAY ferme la fenêtre*

```
243         if (boutonFenetre == 1)   // bloqué en AWAY ou s'il pleut, sinon il ouvre ou ferme
244           {
245             boutonFenetre = 0;
246             if (mode == AWAY || pluie == 1)
247                 envoyer("Fenetre bloquee\r\n");
248             else if (fenetreOuverte == 1)
249                 fermerFenetre();
250             else
251                 ouvrirFenetre();
252           }
```

Extrait du main.c, lignes 243 à 252

Dès que la valeur de pluie passe à ***4095****, le système affiche* ***RAIN****, ferme la fenêtre et refuse les appuis sur le bouton. Une fois le capteur sec, le bouton fonctionne de nouveau.*

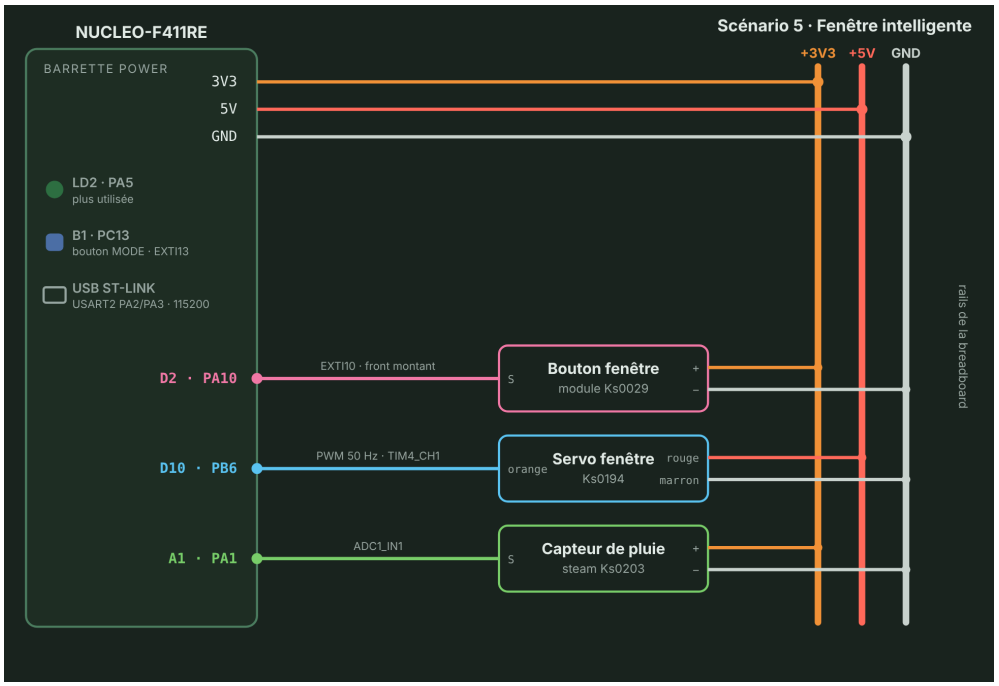


Figure 6 : Câblage du scénario 5 (servo, bouton fenêtre, capteur de pluie)

```
Gaz = 589 Pluie = 349
Gaz = 590 Pluie = 4095
RAIN
WINDOW CLOSED
...
Fenetre bloquee
Fenetre bloquee
...
Gaz = 630 Pluie = 501
Plus de pluie
...
WINDOW OPEN
```

Figure 7 : Terminal série, pluie détectée, fenêtre fermée puis bloquée, puis déblocage

Problème rencontré:

Ce scénario nous a demandé le plus de débogage. Le servo émettait de petits « tics » réguliers sans tourner. Un premier servo était défectueux et nous l'avons remplacé, puis le même symptôme est revenu à cause d'une alimentation 5V insuffisante, que nous avons corrigée. Le code n'était pas en cause. Les premières valeurs du capteur de pluie étaient aussi incohérentes, à cause d'une entrée flottante due à un mauvais câblage. Enfin, le bouton fenêtre provoque parfois deux bascules pour un seul appui (rebond au relâchement ou parasite au démarrage du servo). Nous pensons que, pour une prochaine itération, relire l'état de la broche avant d'agir corrigerait ce défaut.

2.6 Scénario 6 : mesure de température (TMP102, I²C)

*Ce scénario demandait d'écrire un driver pour le TMP102 à partir de sa datasheet (Texas Instruments, SBOS397), sans bibliothèque, avec au minimum quatre fonctions : **initialiser**, **déclencher une mesure**, **lire** et **convertir**. La température est affichée toutes les 3 s.*

Partie I²C

*L'I²C utilise deux fils, **SCL** (horloge) et **SDA** (données) en drain ouvert : les lignes ne remontent à 1 que grâce à des pull-up. Nous avons utilisé :*

- ***I2C1** en Standard Mode, à **100 kHz***
- ***PB8 = SCL (D15)** et **PB9 = SDA (D14)** en Alternate Function Open Drain avec les pull-up internes, l'autre option pour I2C1 (**PB6**) étant déjà prise par le servo*
- *une alimentation du TMP102 en **3,3V**, sa datasheet la limitant à 3,6V*

En activant I2C1, CubeMX a déplacé SDA sur PB7 et affecté PB9 à I2C2. Nous avons forcé PB9 en I2C1_SDA à la main puis activé les pull-up sur les deux broches.

Adresse et registres du TMP102

*La broche ADD0 du TMP102 est reliée à la masse ce qui donne l'adresse 7 bits **0x48** (datasheet, tableau d'adressage). Sur le bus l'adresse occupe les 7 bits de poids fort du premier octet et le dernier bit indique la lecture ou l'écriture. La HAL attend donc l'adresse décalée d'un bit, 0x48 << 1 = **0x90**, d'où la définition TMP102_ADR dans **USER CODE PD**.*

*Le premier octet écrit après l'adresse est le **registre pointeur**, qui choisit le registre auquel on accède : **0x00** pour la température et **0x01** pour la configuration. Il ne faut pas confondre ce pointeur avec la valeur écrite dans le registre de configuration.*

*Comme demandé lors de la validation, nous détaillons ici la valeur **0x61A0** écrite dans ce registre à l'initialisation, et pourquoi. Ce registre fait 16 bits et vaut **0x60A0** au démarrage. Nous avons choisi le mode **one-shot**, dans lequel le capteur reste en veille et ne mesure que sur demande. Pour cela, nous ne modifions que deux bits de l'octet de poids fort, le second octet (0xA0) restant celui d'origine.*

Octet de poids fort	OS (b7)	R1	R0	F1	F0	POL	TM	SD (b0)	Rôle
0x60 (démarrage)	0	1	1	0	0	0	0	0	conversion continue
0x61 (init)	0	1	1	0	0	0	0	**1**	SD = 1 : mise en veille
0xE1 (mesure)	**1**	1	1	0	0	0	0	**1**	OS = 1 : lance une mesure

*Le bit **SD** (shutdown, bit 0) met le capteur en veille après la conversion en cours : c'est la valeur **0x61A0**, écrite à l'initialisation. Le bit **OS** (one-shot, bit 7) écrit pendant la veille lance une seule conversion la valeur **0xE1A0** est écrite pour déclencher une mesure. R1 et R0 sont en lecture seule et valent 1 et **F1**, **F0**, **POL** et **TM** concernent la fonction thermostat (broche ALERT), que nous n'utilisons pas. Dans le second octet, le bit **EM** = 0 garde le format normal sur 12 bits.*

Nous avons choisi ce mode parce qu'il colle exactement aux quatre fonctions demandées, et parce que le capteur ne consomme presque rien entre deux mesures.

Lecture et conversion

*Le registre de température renvoie deux octets, poids fort en premier. En 12 bits, la valeur est cadrée à gauche. Nous recollons les deux octets puis décalons de 4 bits vers la droite. Chaque unité vaut **0,0625 °C**. En convertissant d'abord en **int16_t**, le décalage conserve le signe des températures négatives (codées en complément à 2). Par exemple, les octets 0x18 et 0x20 donnent 0x1820, soit 0x182 = 386 après décalage, et 386 × 0,0625 = **24,1 °C**.*

*Les quatre fonctions du driver sont dans **USER CODE 0** :*

```c
/* Driver du TMP102, écrit à partir de la datasheet TI (SBOS397) */

void tmp102_init(void)            // met le capteur en veille (bit SD à 1) : 0x60A0 devient 0x61A0
{
  uint8_t trame[3] = {REG_CONFIG, 0x61, 0xA0};
  HAL_I2C_Master_Transmit(&hi2c1, TMP102_ADR, trame, 3, 100);
}

void tmp102_mesurer(void)         // demande une seule mesure (bit OS à 1) : 0xE1A0
{
  uint8_t trame[3] = {REG_CONFIG, 0xE1, 0xA0};
  HAL_I2C_Master_Transmit(&hi2c1, TMP102_ADR, trame, 3, 100);
}

int16_t tmp102_lire(void)         // lit les 2 octets du registre de température
{
  uint8_t pointeur = REG_TEMP;
  uint8_t octets[2] = {0, 0};
  HAL_I2C_Master_Transmit(&hi2c1, TMP102_ADR, &pointeur, 1, 100);  // on pointe d'abord sur le registre de température
  HAL_I2C_Master_Receive(&hi2c1, TMP102_ADR, octets, 2, 100);      // puis on lit l'octet de poids fort et celui de poids faible
  return (int16_t)((octets[0] << 8) | octets[1]) >> 4;             // on garde les 12 bits utiles, le signe est conservé
}

float tmp102_convertir(int16_t brut)
{
  return brut * 0.0625f;          // 1 bit = 0,0625 °C ; le f évite un calcul en double
}
```

Extrait du main.c, lignes 112 à 138

*Au démarrage (**USER CODE 2**), nous vérifions que le capteur répond avec HAL_I2C_IsDeviceReady() puis nous appelons tmp102_init() et lançons une première mesure. Ensuite, à chaque passage de TIM11 (**USER CODE 3**) nous lisons la mesure lancée 3s plus tôt avant de relancer la suivante : le programme n'attend jamais la fin d'une conversion. Comme la version allégée de la bibliothèque C de STM32CubeIDE ne gère pas %f dans sprintf(), nous affichons la température en dixièmes avec des entiers.*

```c
            temperature = tmp102_convertir(tmp102_lire());  // on lit la mesure lancée il y a 3 s
            tmp102_mesurer();                               // et on relance la suivante
            int t10 = (int)(temperature * 10);              // sprintf ne gère pas %f : 23,5 °C devient 235
            sprintf(texte, "Temp = %d.%d C\r\n", t10 / 10, t10 % 10);
            envoyer(texte);
```

Extrait du main.c, lignes 281 à 285

D'après la datasheet, chaque étape du driver produit la trame suivante sur le bus (S = start, A = acquittement, N = non-acquittement du maître, P = stop) :

Étape	*Trame sur SDA*
Initialisation	*S · 0x90 · A · 0x01 · A · 0x61 · A · 0xA0 · A · P*
Déclenchement	*S · 0x90 · A · 0x01 · A · 0xE1 · A · 0xA0 · A · P*
Choix du registre température	*S · 0x90 · A · 0x00 · A · P*
Lecture des deux octets	*S · 0x91 · A · MSB · A · LSB · N · P*

À 100 kHz, un bit dure 10 µs, ce qui fait 90 µs par octet en comptant le bit de réponse qui le suit. Pendant la lecture, les rôles s'inversent : c'est le STM32 qui reçoit, il répond donc « reçu » (ACK) après le premier octet puis « stop » (NACK) après le second, pour dire au capteur qu'il a tout lu.

*Au démarrage, le terminal affiche **TMP102 OK**. La température ambiante est stable vers **22°C** et monte jusqu'à **27,6 °C** quand on pose le doigt sur le capteur.*

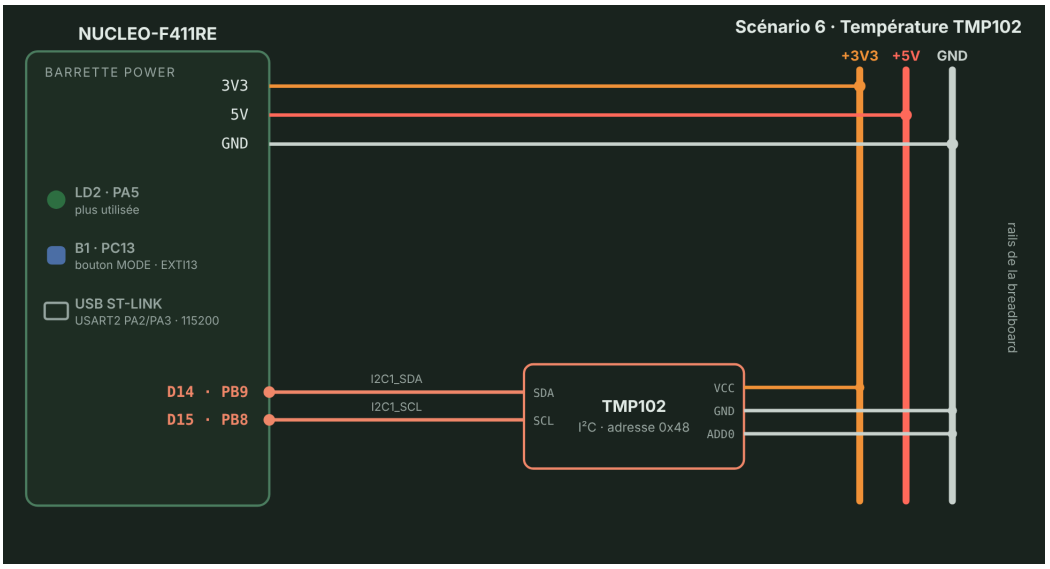


Figure 8 : Câblage du scénario 6 (TMP102 sur I2C1)

```
Temp = 21.9 C
Temp = 22.1 C
Temp = 22.3 C
Temp = 25.8 C
Temp = 26.4 C
Temp = 26.8 C
Temp = 27.3 C
Temp = 27.6 C
Temp = 27.2 C
Temp = 26.9 C
```

Figure 9 : Terminal série, température lue toutes les 3 s, doigt posé sur le capteur

Nous n'avons pas implémenté l'option de gestion automatique de la fenêtre par la température dû à un manque de temps. Nous aurions ouvert la fenêtre au-dessus d'environ 26 °C et l'aurions refermée sous 24 °C (deux seuils, comme pour le gaz) seulement en HOME et sans pluie. La pluie et le mode AWAY seraient restés prioritaires, et un appui sur le bouton aurait suspendu l'automatisme.

2.7 Scénario 7 : intégration du système

Le dernier scénario consistait à réunir toutes les fonctions dans une seule application avec des décisions qui restent cohérentes quand plusieurs événements arrivent en même temps. Comme nous avons développé tous les scénarios dans le même projet et le même main.c depuis le début l'intégration s'est faite au fur et à mesure. Une seule boucle traite un drapeau par source d'événement donc, si un événement arrive pendant un traitement, son drapeau reste à 1 et il est traité au tour suivant.

Le travail propre à ce scénario a surtout consisté à fixer des règles de priorité, résumées sur la Figure 10 :

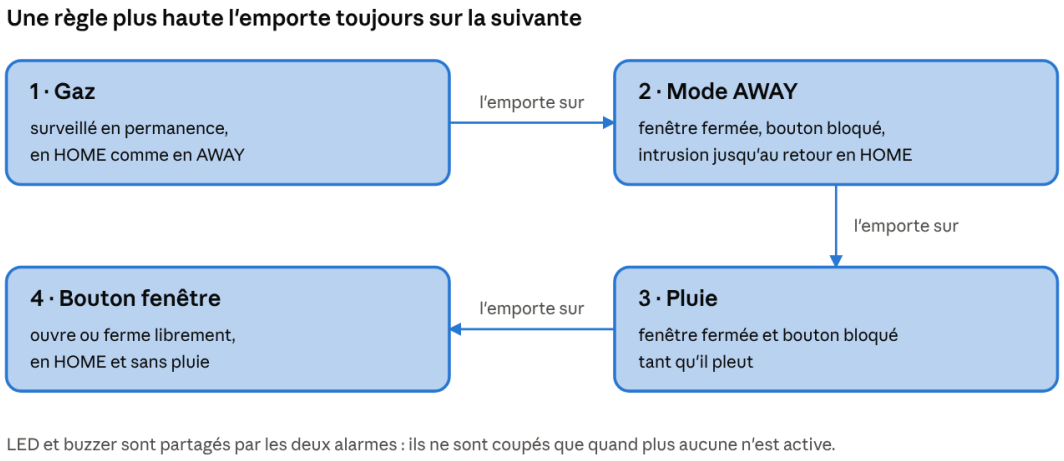


Figure 10 : Règles de priorité de l'application

Par exemple, si une alarme gaz et une intrusion sont actives et que l'on repasse en HOME, l'intrusion est désarmée mais le buzzer et la LED continuent, car l'alarme gaz est toujours là.

Lors de la démonstration, nous avons enchaîné tous les scénarios sans redémarrer la carte, et toutes les fonctions ont cohabité sans conflit. La Figure 11 montre le câblage complet de la maquette.

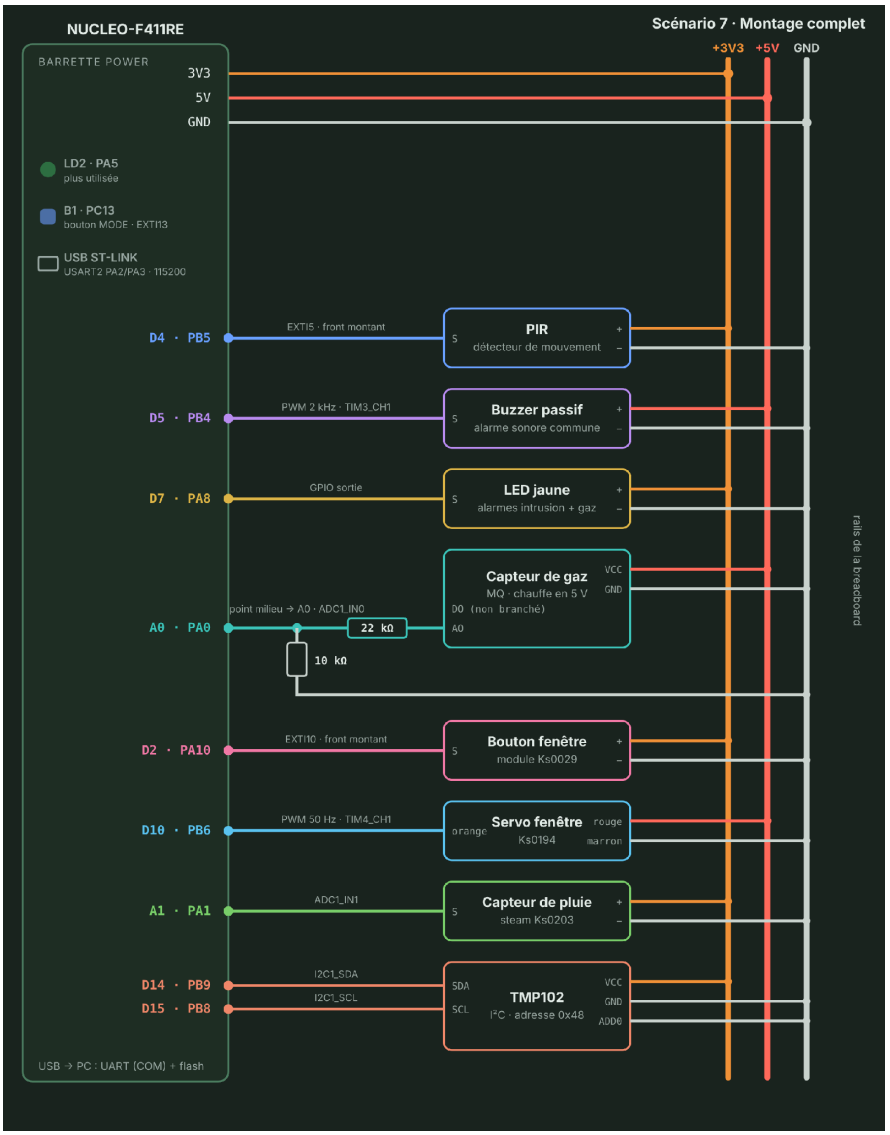


Figure 11 : Câblage complet de la maison (scénario 7)

Conclusion

Ce mini-projet nous a permis d'utiliser dans une même application la plupart des périphériques du STM32 vus en cours (timers en base de temps et en PWM, interruptions externes, ADC en mode scan, UART et I²C). Pour chaque composant, nous avons suivi la même démarche : comprendre son fonctionnement grace au wiki fourni dans le scénario, configurer le bon périphérique en justifiant chaque valeur, programmer le fonctionnement du scénario, puis vérifier sur la maquette. Le fait d'avoir travaillé dans un seul main.c avec des drapeaux nous a permis d'ajouter les sept scénarios un par un sans tout réécrire.